

Binary Trees: The First Branching Structure

Programming and Data Structures — Week 7, Lecture 1

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to define a binary tree precisely in terms of nodes, children, and the recursive structure it shares with the linked lists covered in Week 4; implement a `TreeNode` class and construct a small tree by hand; implement and trace all three standard depth-first traversal orders — preorder, inorder, and postorder — on a concrete example; and state the relationship between a binary tree's height and its size, understanding why this relationship becomes central to the next several lectures.

Outline

Today covers a family-tree analogy for branching structure; the precise step from Week 4's linked list to a binary tree, via one additional reference per node; constructing a small tree by hand; all three depth-first traversal orders, traced completely on a concrete example; the relationship between a tree's height and its size, and why this relationship matters for the lectures that follow; a direct visual comparison between balanced and unbalanced trees holding the same elements; the MTech-track connection to expression trees, tying back to Week 3's stack-based expression evaluation; an in-class quiz; and a summary.

The family-tree analogy

Picture a simplified family-tree diagram, where each person shown can have at most two children displayed beneath them, and each of those children can themselves have their own two children, and so on. Every structure covered in this course through Week 6 — arrays, stacks, queues, and linked lists — is fundamentally linear: conceptually, one element follows another, whether accessed by index or by following a single `next` reference. A binary tree is the first structure in this course with a genuinely branching shape, where a single node can lead to two entirely separate substructures rather than one single successor.

From linked list to tree: one more reference

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.left = None      # reference to the left child, or None
        self.right = None     # reference to the right child, or None
```

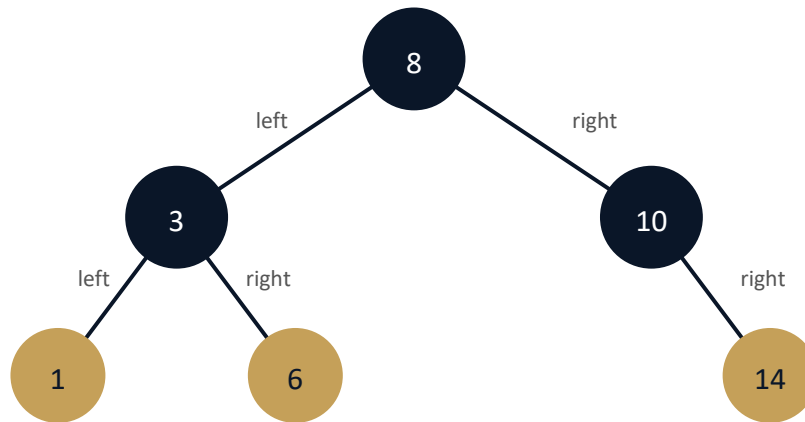
Compare this `TreeNode` directly against Week 4's `Node` class: both hold a `value`, but where a linked list's node had a single `next` reference, a tree node has two, `left` and `right`. In fact, a linked list can be seen as a special, degenerate case of a binary tree — one where every node happens to use only its `right` child (or only its `left`), never branching. This is structurally the same idea as Week 4's step from singly to doubly linked lists — adding one more reference per node to unlock new capability — applied here to create branching rather than bidirectional traversal.

Building a tree by hand

```
root = TreeNode(8)
root.left = TreeNode(3)
root.right = TreeNode(10)
root.left.left = TreeNode(1)
root.left.right = TreeNode(6)
root.right.right = TreeNode(14)
```

Building this six-node tree by hand mirrors exactly how Week 4 built linked lists node by node. `root` serves as the single entry point, exactly analogous to a linked list's head reference — every other node in the tree is reachable only by following some specific path of `left` and `right` references starting from `root`. A node with no children at all, such as the nodes holding 1, 6, and 14 in this example, is called a leaf — the tree's terminal points, playing a role somewhat analogous to a linked list's final node whose `next` is `None`.

The picture: this tree, drawn out



Leaves (gold): nodes with no children. Root at top, branching downward via left/right references.

Drawing the tree built on the previous slide makes its branching structure immediately visible: root 8 branches to 3 (left) and 10 (right); 3 further branches to 1 (left) and 6 (right); 10 branches only to 14 (right), with no left child. This asymmetry — some nodes having one child, some two, some none — is entirely normal for a binary tree in general, unlike the highly regular structure Week 8’s heaps will impose.

Preorder traversal: visit, then left, then right

```
def preorder(node, result=None):
    if result is None:
        result = []
    if node is not None:
        result.append(node.value)      # VISIT first
        preorder(node.left, result)   # then left subtree
        preorder(node.right, result)  # then right subtree
    return result
```

Preorder traversal visits the current node first, before recursing into either subtree — root, then the entire left subtree (recursively, in preorder), then the entire right subtree (recursively, in preorder). On the six-node tree built earlier, this produces [8, 3, 1, 6, 10, 14]: the root 8 first, then everything in the left subtree rooted at 3 (itself preorder: 3, then 1, then 6), then everything in the right subtree rooted at 10 (preorder: 10, then 14).

Inorder traversal: left, then visit, then right

```
def inorder(node, result=None):
    if result is None:
        result = []
    if node is not None:
        inorder(node.left, result)    # left subtree first
        result.append(node.value)     # then VISIT
        inorder(node.right, result)   # then right subtree
    return result
```

Inorder traversal recurses into the left subtree completely first, then visits the current node, then recurses into the right subtree. On the same tree, this produces [1, 3, 6, 8, 10, 14] — notice this is exactly the values in sorted order. This is not a coincidence specific to this example: this particular tree happens to satisfy the binary search tree property (Lecture 2), and inorder traversal of any binary search tree always produces sorted output, a fact that becomes central to next lecture.

Postorder traversal: left, then right, then visit

```
def postorder(node, result=None):
    if result is None:
        result = []
    if node is not None:
        postorder(node.left, result)    # left subtree first
        postorder(node.right, result)   # then right subtree
        result.append(node.value)       # then VISIT last
    return result
```

Postorder traversal recurses into both the left and right subtrees completely before visiting the current node itself, producing [1, 6, 3, 14, 10, 8] on this tree — the root, 8, appears last. Postorder is the natural traversal whenever a node's children must be fully processed before the node itself — safely deleting an entire tree node by node without ever accessing an already-freed child, or evaluating an expression tree (this lecture's MTech addition) from the leaves upward, are both natural postorder applications.

Tracing preorder completely, step by step

```
preorder(8):  visit 8 → result=[8]
```

```

preorder(3): visit 3 → result=[8,3]
  preorder(1): visit 1 → result=[8,3,1]
    preorder(None-left): nothing
    preorder(None-right): nothing
  preorder(6): visit 6 → result=[8,3,1,6]
preorder(10): visit 10 → result=[8,3,1,6,10]
  preorder(None-left): nothing
  preorder(14): visit 14 → result=[8,3,1,6,10,14]

```

Tracing preorder completely, call by call, shows the recursion visiting a node immediately upon entering its call, then recursing left fully before recursing right — exactly mirroring the recursive call-stack tracing technique from Week 3's call-stack lecture and Week 4's linked-list traversal, now applied to a branching structure. The final result, [8, 3, 1, 6, 10, 14], matches the earlier claim exactly.

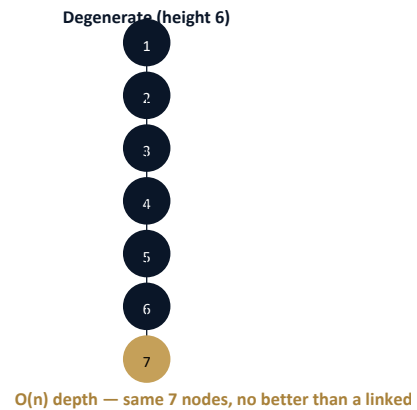
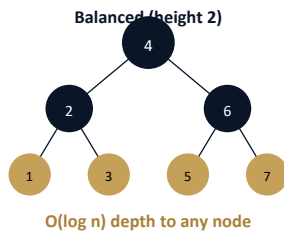
Height and size: the relationship that matters most

Two quantities matter above all others for a binary tree: its size n , the total number of nodes, and its height h , the length of the longest path from the root down to any leaf. A binary tree of height h can hold at most $2^{h+1} - 1$ nodes, achieved when every level is completely full — this is the exact same doubling pattern as Week 5 Lecture 4's decision tree, where a tree of height h had at most 2^h leaves. Equivalently, rearranging this relationship, a tree holding n nodes must have height at least $\log_2(n + 1) - 1$ — but only if the tree happens to be well-balanced. This “if” is precisely what the next several lectures investigate.

Why this relationship is the whole point of the next few lectures

If a tree holding n nodes has height $\Theta(\log n)$, any operation whose cost depends on height — searching, inserting, deleting, as Lecture 2 covers — costs only $O(\log n)$, a dramatic improvement over Week 4's linked list. But nothing in the plain definition of a binary tree given in this lecture *guarantees* this favorable shape at all — a tree can be constructed in a way that degenerates toward one long chain, where every node has only one child. Such a degenerate tree has height $\Theta(n)$, offering no advantage whatsoever over a plain linked list. This exact tension — a tree's *potential* for $O(\log n)$ operations, versus the very real risk of degenerating to $O(n)$ — is the central problem the rest of this week's lectures address.

The picture: balanced vs. degenerate, same 7 nodes



Both trees in this figure hold the exact same seven values. The balanced tree has height 2, giving $O(\log n)$ -depth access to any node. The degenerate chain has height 6, giving no better than $O(n)$ -depth access — identical content, dramatically different performance, determined entirely by how the tree happened to be built.

MTech addition — expression trees, connecting back to Week 3

```
#      +
#     / \
#    3  *
#     / \
#    4  2
root = TreeNode('+')
root.left = TreeNode(3)
root.right = TreeNode('*')
root.right.left = TreeNode(4)
root.right.right = TreeNode(2)
```

An expression tree places operators at internal nodes and operands at leaves, with this example representing $3 + (4 \times 2)$. Traversing this tree in postorder produces exactly 3, 4, 2, *, + — precisely the postfix notation covered in Week 3 Lecture 4's expression-evaluation lecture. This is not a coincidence: postorder naturally visits both operands of an operator before the operator itself, exactly the order Week 3's stack-based postfix evaluator needed to process tokens correctly. Trees and the stack-based algorithms from several weeks ago are two different views of the same underlying structure.

Level-order traversal: a fourth option, using a queue

```
from collections import deque

def level_order(root):
    if root is None:
        return []
    result, q = [], deque([root])
    while q:
        node = q.popleft()
        result.append(node.value)
        if node.left: q.append(node.left)
        if node.right: q.append(node.right)
    return result
```

Level-order traversal is a fourth option, structurally different from the other three: where preorder, inorder, and postorder are all depth-first, relying on recursion (and therefore the call stack, per Week 3), level-order is breadth-first, visiting the tree one full level at a time. It directly reuses Week 3's queue: every node's children are enqueued as it is dequeued and visited, guaranteeing nodes are visited in order of increasing depth. On this lecture's example tree, level-order produces [8, 3, 10, 1, 6, 14] — root first, then both of its children, then all four grandchildren-level nodes, visibly grouped by depth in a way none of the depth-first orders are.

Exercise

As an exercise, build the expression tree for $(5 - 2) \times (3 + 4)$ by hand, placing $-$ and $+$ as internal nodes with the appropriate operand leaves, and $*$ as the root. Trace its postorder traversal and confirm it produces the correct postfix form, connecting back to Week 3 Lecture 4's shunting-yard algorithm. Separately, implement `tree_height(node)` recursively — a node's height is 1 plus the maximum of its children's heights, with a `None` node conventionally having height -1 — and confirm it returns 2 for this lecture's six-node example tree.

All four traversals, side by side, on one tree

Consolidating all four traversals on the same six-node tree makes their differences concrete rather than abstract: the same six values, in four genuinely different orders, each useful for a different purpose — inorder for sorted output on a binary search tree (Lecture 2), postorder for safe deletion or expression evaluation, preorder for reconstructing a tree's structure (an exercise for a later course), and level-order for anything requiring depth-grouped processing, such as printing a tree level by level.

Quiz — trace the traversal

For a tree with root 5, whose left child is 2 (which itself has a left child of 1), and whose right child is 8, work out what inorder traversal produces, before checking the answer.

Quiz — answer

Inorder traversal produces $[1, 2, 5, 8]$. The left subtree rooted at 2 (which itself has left child 1) is fully traversed in inorder first, giving $[1, 2]$ — 1 before 2, since 1 is 2's left child. Then the root, 5, is visited. Then the right subtree, just the single node 8, is traversed. Notice this result is again in sorted order — the same observation made earlier in this lecture, previewing Lecture 2's binary search tree property.

Summary

A binary tree extends Week 4's linked-list node with a second reference, `left` and `right`, which is the single structural change that introduces genuine branching into this course for the first time. Preorder, inorder, and postorder are the three standard depth-first traversal orders, distinguished entirely by when each node is visited relative to its two children. A tree of height h can hold at most $2^{h+1} - 1$ nodes, but nothing in a binary tree's plain definition guarantees this favorable shape — a degenerate tree can have height $\Theta(n)$, no better than Week 4's linked list. This tension is the central problem the remainder of this week addresses. Next lecture introduces the binary search tree property, which turns this branching structure into a genuinely efficient search mechanism, connecting directly back to Week 1's binary search.